

Hybrid Domain-Adaptive Framework for Cross-System Log Anomaly Detection

Teja I^{1,*}

¹Department of CSE
PES University
Bangalore, India
tejal@pes.edu

Spoorthi V Pai^{2,†}

²Department of EEE
PES University
Bangalore, India
spoorthipai23@gmail.com

Lahari L M^{1,‡}

¹Department of CSE
PES University
Bangalore, India
laharik024@gmail.com

Abstract—Modern computing infrastructures — including distributed systems, cloud platforms, Linux servers, enterprise operating systems, and supercomputers — generate massive volumes of runtime logs during execution. These logs encode valuable information about authentication events, abnormal system behavior, hardware and software failures, unauthorized access attempts, and critical security incidents. Given the exponential growth in log volume, manual inspection has become highly inefficient and practically infeasible, necessitating intelligent and scalable automated anomaly detection systems.

This paper presents a comprehensive study and implementation of a *unified hybrid and domain-adaptive framework* for the detection of log anomalies between systems. The framework integrates five complementary approaches: (i) machine learning-based sequence detection inspired by DeepLog, (ii) rule-based cybersecurity detection for known attack signatures, (iii) a hybrid detection layer that fuses the outputs of (i) and (ii), (iv) Transformer-based sequential learning for contextual log understanding, and (v) domain-adaptive Transformer architectures that leverage Gradient Reversal Layers (GRL) to achieve cross-system generalization.

Experiments were conducted on four heterogeneous datasets: the Hadoop Distributed File System (HDFS) log data set, the Blue Gene/L (BGL) supercomputer data set, the Windows event log data set and real-world Linux authentication logs. Evaluation was performed using Accuracy, Precision, Recall, F1-score, ROC curves, and Confusion Matrices. Results demonstrate that conventional machine learning models suffer from limited recall under unseen anomalies, while rule-based systems achieve high precision but poor adaptability. The proposed hybrid and domain-adaptive approaches substantially improve detection robustness, with the GRL-augmented Transformer achieving F1-scores up to 0.97 in cross-domain transfer scenarios.

Index Terms—Log Anomaly Detection, DeepLog, Transformer, Gradient Reversal Layer (GRL), Domain Adaptation, Linux Authentication Logs, Cybersecurity, Machine Learning, Hybrid Detection, Sequence Learning

I. INTRODUCTION

Modern computing infrastructures continuously emit runtime logs that serve as a first-line source of operational telemetry. Distributed file systems, supercomputing clusters, enterprise operating systems, cloud-native services, and embedded Linux servers each produce log streams capturing user authentication activities, process execution traces, network communication events, kernel operations, administrative commands, and security-related anomalies. As infrastructure scales, these log volumes have grown from thousands of entries per day

to billions, rendering traditional manual inspection entirely impractical [1].

Conventional rule-based intrusion detection and log monitoring systems rely on statically defined signatures and threshold conditions. Although effective for catalogued and well-characterised threat patterns, such systems suffer from a fundamental brittleness: they cannot identify previously unseen or zero-day anomalies, and their signature libraries require continuous manual maintenance. As threat landscapes evolve and infrastructure heterogeneity increases, the limitations of purely rule-driven approaches become increasingly apparent.

Machine learning and deep learning techniques have therefore emerged as promising alternatives for automated, data-driven log anomaly detection. Sequence learning approaches such as DeepLog [1] demonstrated that system log events exhibit strong temporal dependencies that can be effectively modelled using Long Short-Term Memory (LSTM) networks. Subsequent research efforts introduced quantitative and semantic feature integration (LogAnomaly [2]), BERT-inspired masked event prediction for log understanding (LogBERT [3]), and full Transformer architectures [5] for capturing long-range contextual relationships within event sequences.

Despite these advancements, one critical and largely unresolved challenge persists: *domain shift*. A model trained exclusively on one operational environment commonly exhibits severe performance degradation when deployed in a different environment, because distinct systems generate logs with different formats, event distributions, semantic structures, and temporal behavioural patterns. For example, HDFS block-operation logs are structurally and semantically distinct from Windows security event logs or Linux PAM authentication logs. Without explicit mechanisms to bridge this gap, trained models overfit to domain-specific idiosyncrasies and fail to generalize.

This paper addresses the problem of cross-system log anomaly detection through a comprehensive hybrid and domain-adaptive framework. The specific contributions of this work are as follows:

- 1) A practical Linux-based anomaly detection system that combines frequency-based sequence learning with manually encoded cybersecurity rules for identifying brute-force login attacks, user enumeration, and privilege escalation.

- 2) A hybrid detection framework that fuses machine learning predictions with rule-based alerts to achieve balanced precision and recall across heterogeneous anomaly types.
- 3) A Transformer-based sequential learning architecture for contextual log event modelling applied to HDFS, BGL, and Windows datasets.
- 4) A domain-adaptive Transformer integrating Gradient Reversal Layers (GRL) to learn domain-invariant representations, enabling effective cross-system transfer without retraining.
- 5) Systematic comparative evaluation across all models using standard classification metrics and cross-domain transfer experiments.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the datasets used. Section IV presents the overall system architecture. Section V details the methodology for each component. Section VI elaborates the domain-adaptive Transformer framework. Section VII presents experimental results and analysis. Section VIII provides an extended discussion of findings. Section IX acknowledges limitations. Section X outlines future directions, and Section XI concludes the paper.

II. RELATED WORK

A. DeepLog

Du et al. [1] proposed DeepLog, one of the earliest deep learning frameworks specifically designed for log anomaly detection. DeepLog employs LSTM networks to model normal system behaviour as a sequence of log keys. Given a history of k log events, the model predicts the probability distribution over the next expected event. Events that fall below a defined probability threshold are flagged as anomalous. DeepLog was evaluated on HDFS logs and demonstrated competitive detection accuracy with low false-positive rates. However, its design is inherently single-domain, and its dependency on a fixed vocabulary limits generalizability.

B. LogAnomaly

Meng et al. [2] extended sequence-based log anomaly detection by incorporating both sequential and quantitative log features. LogAnomaly employs synonym and antonym analysis of log templates to construct semantic log vectors, and then applies a neural network to detect anomalies at the semantic level. The use of quantitative features (i.e., event frequency counts within a workflow) further improves detection in scenarios where sequential ordering alone is insufficient.

C. LogBERT

Guo et al. [3] introduced LogBERT, adapting the BERT masked language modelling pre-training strategy to the log analysis domain. Log keys are treated as tokens, and the model is pre-trained by randomly masking tokens and learning to reconstruct them. During inference, unusually low reconstruction probabilities indicate anomalous sequences. LogBERT demonstrated improved contextual understanding compared to LSTM-based methods by capturing bidirectional event dependencies.

D. Domain-Adversarial Neural Networks (DANN)

Ganin et al. [4] introduced the concept of domain-adversarial training through the inclusion of a Gradient Reversal Layer (GRL) within the neural network architecture. The GRL reverses gradients during backpropagation at the domain classifier head, thereby penalizing feature representations that are discriminative with respect to domain identity. This forces the shared feature extractor to produce domain-invariant representations, enabling robust transfer across domains without requiring target-domain labelled data.

E. Attention Mechanisms and Transformers

Vaswani et al. [5] introduced the Transformer architecture, replacing recurrence with multi-head self-attention to model dependencies between arbitrary positions in a sequence with constant computational depth. Transformers have since become the dominant architecture in natural language processing tasks and have increasingly been adopted for time-series, event-sequence, and log analysis applications due to their superior long-range dependency modelling capabilities.

F. Summary and Research Gap

While prior work has individually addressed sequence modelling, semantic log analysis, and domain-adversarial transfer, no unified framework has systematically combined cybersecurity-inspired rule-based detection with machine learning adaptability and GRL-based cross-system domain adaptation evaluated across four heterogeneous log datasets. This work fills that gap.

III. DATASETS

A. HDFS Dataset

The Hadoop Distributed File System (HDFS) log dataset is one of the most widely-used benchmark corpora in log anomaly detection research [6]. The dataset was collected from a Hadoop cluster deployed on Amazon EC2 nodes and comprises log messages associated with block-level operations. Each log entry contains a timestamp, a log level, a component identifier, a block identifier, and a free-text message describing the operation performed.

For this work, the HDFS dataset was used primarily for studying sequential anomaly detection concepts, preprocessing strategies, event encoding, sequence generation, and initial Transformer training experiments. A total of 11,175,629 raw log lines are available, of which 16,838 block sequences are labeled as anomalous and 558,223 are normal.

B. BGL Dataset

The Blue Gene/L (BGL) supercomputer log dataset [7] was collected from a large-scale supercomputing system at Lawrence Livermore National Laboratory. It contains approximately 4,747,963 log messages recorded across failure and normal operational periods. Each entry includes a timestamp, node identifier, log level (ranging from FATAL to INFO), and a variable-length message string. The dataset is characterized by high event diversity, long tail distributions of rare events,

and complex failure cascades spanning multiple system components.

The BGL dataset was used in this work for evaluating the cross-domain generalization capability of trained Transformer models. Since BGL logs differ significantly from HDFS logs in vocabulary, event distribution, and sequence structure, the dataset provides a challenging benchmark for domain shift assessment.

C. Windows Event Log Dataset

The Windows event log dataset was incorporated to further increase the heterogeneity of the experimental evaluation. Windows Security and System event logs contain authentication events, administrative operations, system notifications, service lifecycle events, and security-related activities logged by the Windows Event Logging subsystem. Unlike the line-structured HDFS and BGL datasets, Windows logs are structured using XML-based schemas, requiring separate parsing strategies.

This dataset was particularly useful for evaluating whether the proposed domain-adaptive Transformer framework could learn generalised representations that are genuinely independent of system-specific log structure and event semantics.

D. Linux Authentication Log Dataset

Linux authentication logs — specifically `auth.log` files and the publicly available `Linux_2k` dataset from the LogHub repository [6] — were used for the practical implementation component of this project. These logs record PAM authentication events, SSH session management, `sudo` command executions, failed login attempts, and account management activities. The Linux dataset provided a realistic and practically relevant evaluation environment for cybersecurity anomaly detection, including brute-force attack simulation, user enumeration detection, and privilege escalation identification.

Table I summarises the characteristics of all four datasets.

TABLE I
SUMMARY OF DATASETS USED IN EXPERIMENTS

Dataset	Source	Log Lines	Anomaly Rate	Task
HDFS	Hadoop EC2	~11.2M	2.93%	Seq. Learning, Transfer
BGL	Supercomp.	~4.7M	7.37%	Cross-Domain Target
Windows	Windows OS	~0.5M	~5%	Cross-Domain Target
Linux	Auth. Logs	2,000	Variable	Practical Detection

IV. OVERALL SYSTEM ARCHITECTURE

The proposed framework is organized into seven functional layers, as illustrated in Figure 1.

a) *Layer 1 — Data Collection*: Raw log files are collected from heterogeneous sources including HDFS, BGL, Windows Event Viewer, and Linux `/var/log/auth.log`.

b) *Layer 2 — Unified Log Preprocessing*: Raw logs are parsed using regular-expression templates to extract event identifiers. Timestamps, numerical values, hexadecimal addresses, and domain-specific noise tokens are normalized. A global LabelEncoder maps event templates to integer identifiers shared across all datasets.

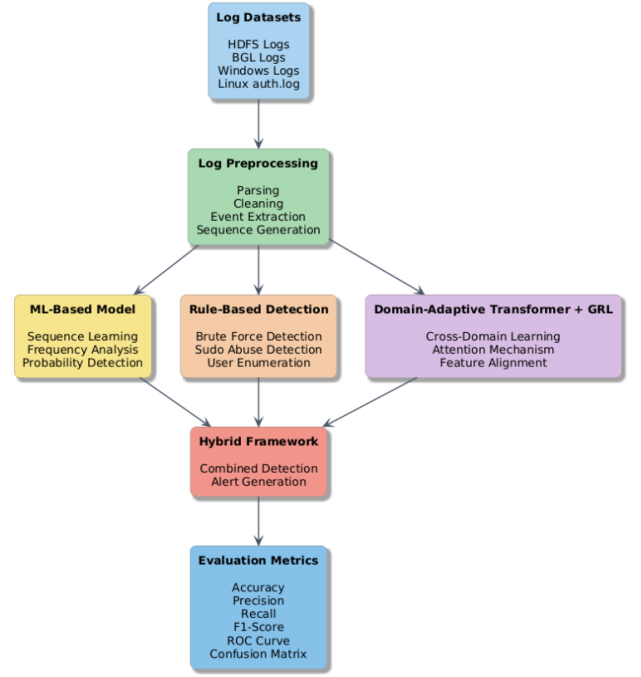


Fig. 1. Overall architecture of the proposed hybrid and domain-adaptive log anomaly detection framework

c) *Layer 3 — Machine Learning Module*: A frequency-based sequence learning model computes conditional probabilities over event n -grams derived from sliding-window segmentation and flags low-probability sequences as anomalous.

d) *Layer 4 — Rule-Based Detection Module*: Manually encoded cybersecurity rules detect known attack signatures including brute-force login attempts (repeated E1 events), user enumeration (repeated E4 events), and privilege escalation (excessive E3 events).

e) *Layer 5 — Domain-Adaptive Transformer Module*: A Transformer encoder equipped with multi-head self-attention processes encoded event sequences. A GRL-augmented domain adaptation head forces domain-invariant representation learning.

f) *Layer 6 — Hybrid Detection Framework*: The outputs of the ML module and the rule-based module are fused using a logical OR policy. A sequence is flagged anomalous if either module raises an alert.

g) *Layer 7 — Evaluation and Visualization*: All model outputs are evaluated using Accuracy, Precision, Recall, F1-score, ROC curves, and Confusion Matrices.

V. METHODOLOGY: LINUX-BASED LOG ANOMALY DETECTION

A. Log File Loading and Preprocessing

Linux authentication log files are loaded in UTF-8 encoding with permissive error handling to accommodate encoding irregularities commonly present in real-world system logs.

```

with open("Linux_2k.log", "r",
          encoding="utf-8", errors="ignore")
    ↪ as f:
  
```

```
logs = f.readlines()
print(f"Total log lines loaded: {len(logs)}")
```

Listing 1. Loading Linux authentication logs.

B. Log Parsing and Event Abstraction

Raw log messages are mapped to a compact vocabulary of structured event identifiers using keyword-based pattern matching. This abstraction reduces textual complexity and enables uniform sequential modelling across heterogeneous log formats. The event taxonomy is defined as:

- **E0** — Unclassified or background event.
- **E1** — Failed password authentication attempt.
- **E2** — Accepted (successful) password authentication.
- **E3** — sudo privilege elevation command.
- **E4** — Invalid user login attempt.

```
def parse_event(line: str) -> str:
    """Map a raw log line to a structured
    ↪ event identifier."""
    line = line.lower()
    if "failed password" in line:
        return "E1"
    elif "accepted password" in line:
        return "E2"
    elif "sudo" in line:
        return "E3"
    elif "invalid user" in line:
        return "E4"
    else:
        return "E0"

events = [parse_event(line) for line in logs]
print(f"Event stream length: {len(events)}")
```

Listing 2. Event abstraction from raw log messages.

Figure 2 illustrates the complete parsing and preprocessing workflow.

Fixed-length event sequences are extracted from the linearised event stream using a sliding window of width w . Each window position i generates an input sequence $s_i = [e_i, e_{i+1}, \dots, e_{i+w-1}]$. For a window size $w = 3$, an example event stream:

$$E_2 \rightarrow E_3 \rightarrow E_1 \rightarrow E_1 \rightarrow E_4 \rightarrow E_2 \rightarrow E_3$$

yields the following sequences:

$$[E_2, E_3, E_1], [E_3, E_1, E_1], [E_1, E_1, E_4], [E_1, E_4, E_2], \dots$$

```
def create_sequences(events: list, window: int
    ↪ = 3) -> list:
    """Generate overlapping sliding-window
    ↪ event sequences."""
    return [events[i:i + window]
            for i in range(len(events) -
    ↪ window)]

sequences = create_sequences(events, window=3)
print(f"Total sequences generated: {len(
    ↪ sequences)}")
```

Listing 3. Sliding-window sequence generation.

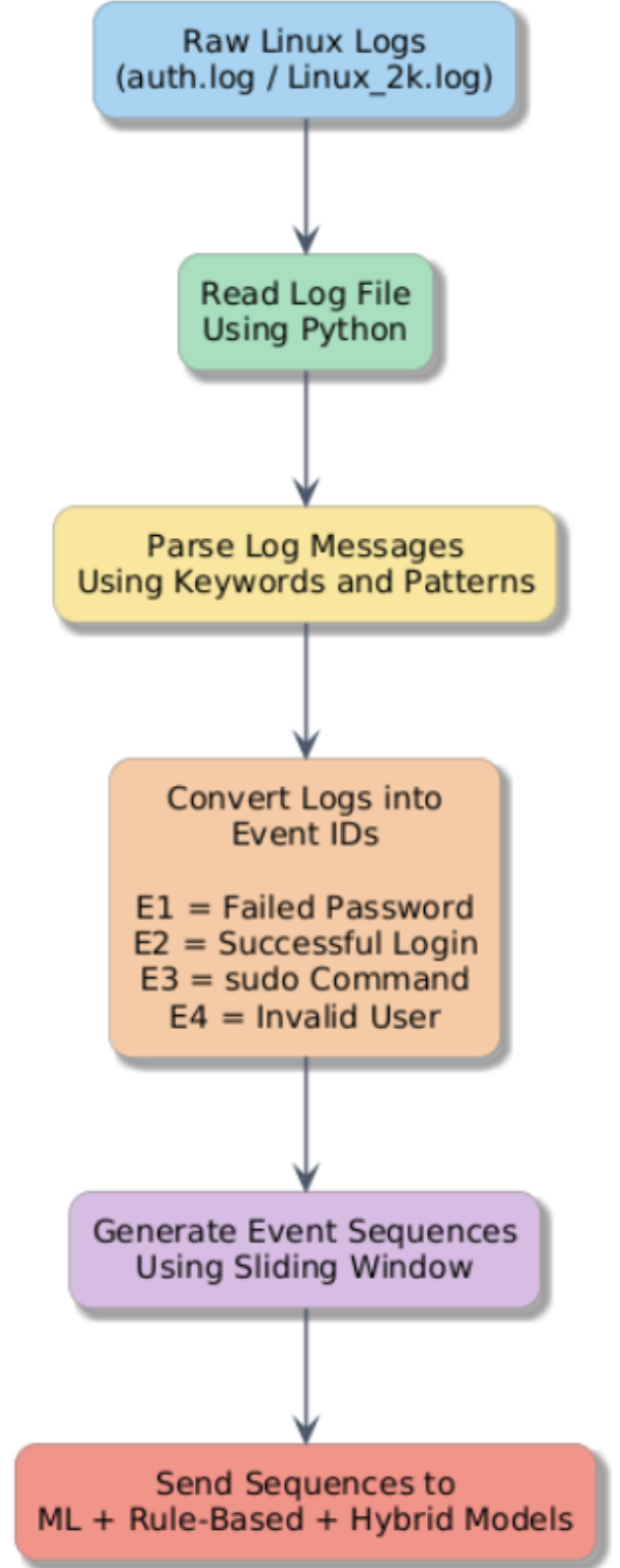


Fig. 2. Linux log parsing and preprocessing workflow for anomaly detection, from raw log lines through event abstraction to structured event sequences

Figure 3 illustrates the sliding-window mechanism.

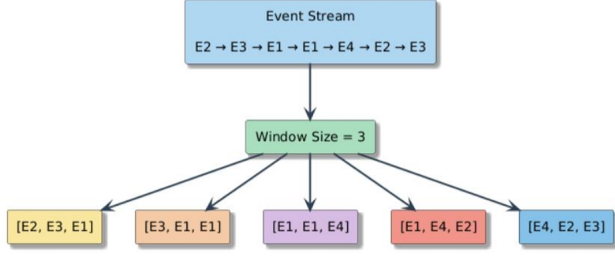


Fig. 3. Sliding window-based sequence generation from parsed log events. Each window of width $w = 3$ yields one training or inference sequence

C. Machine Learning-Based Anomaly Detection

The ML module learns a probability distribution $P(s)$ over observed sequences from the training corpus. Given a sequence s , the estimated probability is:

$$\hat{P}(s) = \frac{c(s)}{\sum_{s'} c(s')}$$

where $c(s)$ denotes the observed frequency of sequence s in the training data. A sequence is flagged as anomalous if its estimated probability falls below a threshold τ :

$$\text{anomalous}(s) = \begin{cases} 1 & \text{if } \hat{P}(s) < \tau \\ 0 & \text{otherwise} \end{cases}$$

```
from collections import defaultdict

# Build sequence frequency model
model = defaultdict(int)
for seq in sequences:
    model[tuple(seq)] += 1

# Normalize to probabilities
total = sum(model.values())
model = {k: v / total for k, v in model.items()}

def detect_anomaly(seq: list, model: dict, threshold: float = 0.05) -> bool:
    """Return True if the sequence is anomalous."""
    prob = model.get(tuple(seq), 0.0)
    return prob < threshold
```

Listing 4. Frequency-based sequence probability model.

D. Rule-Based Cybersecurity Detection

The rule-based module encodes domain knowledge from established cybersecurity threat intelligence. Three attack signatures are implemented:

- **Brute-Force Attack:** Five or more consecutive E1 (failed password) events indicate a systematic password-guessing campaign against a target account or service.

- **User Enumeration:** More than five E4 (invalid user) events suggest an adversary is probing for valid account names.
- **Privilege Escalation:** More than five E3 (sudo) events within a session suggests abnormal administrative activity that may indicate lateral movement or insider threat behaviour.

```
def rule_engine(events: list) -> list:
    """
    Apply cybersecurity rules to generate threat alerts.

    Parameters
    -----
    events : list of str
        Parsed event stream.

    Returns
    -----
    alerts : list of str
        Triggered alert descriptions.
    """
    alerts = []

    # Rule 1: Brute-force 5+ consecutive failed passwords
    consecutive_fails = 0
    for event in events:
        if event == "E1":
            consecutive_fails += 1
            if consecutive_fails >= 5:
                alerts.append("Brute-Force Attack Detected")
                consecutive_fails = 0  # Reset after alert
        else:
            consecutive_fails = 0

    # Rule 2: User enumeration repeated invalid-user events
    if events.count("E4") > 5:
        alerts.append("User Enumeration Attack Detected")

    # Rule 3: Privilege escalation excessive sudo usage
    if events.count("E3") > 5:
        alerts.append("Excessive Sudo Usage / Privilege Escalation Detected")

    return alerts
```

Listing 5. Cybersecurity rule engine.

E. Hybrid Detection Framework

The hybrid framework aggregates decisions from both the ML and rule-based modules. Formally, for a given event sequence s and the corresponding event stream $\mathcal{E}$:

$$\text{Alert}(s, \mathcal{E}) = \text{ML_Anomaly}(s) \vee |\text{Rules}(\mathcal{E})| > 0$$

This logical-OR fusion ensures that low-probability statistical patterns detected by the ML module are surfaced even when rule conditions are not met, and that deterministic rule matches generate alerts even when frequency-based scores are marginally above threshold. The complementary nature of the two modules results in improved recall without a proportional sacrifice in precision.

VI. DOMAIN-ADAPTIVE TRANSFORMER FRAMEWORK

A. Unified Preprocessing Pipeline

A unified preprocessing pipeline was developed to standardize heterogeneous log datasets from HDFS, BGL, Windows, and Linux environments for effective cross-domain learning. Since each dataset contains different log formats, event vocabularies, and sequence distributions, consistent preprocessing is essential.

The preprocessing pipeline includes the following stages:

- 1) **Regex-Based Parsing:** Dataset-specific regular expressions extract event templates from raw log messages.
- 2) **Noise Normalization:** Timestamps, numeric values, IP addresses, and irrelevant tokens are replaced with generic placeholders to improve consistency across datasets.
- 3) **Global Label Encoding:** A unified `LabelEncoder` assigns consistent integer identifiers to events across all datasets.
- 4) **Sliding-Window Segmentation:** Fixed-length overlapping sequences of length $L = 20$ are generated for model training.
- 5) **Padding and Truncation:** Sequences are padded or truncated to maintain uniform input dimensions for the Transformer encoder.

B. Transformer-Based Sequential Learning Architecture

The Transformer encoder architecture employed in this work consists of five primary components:

- 1) **Token Embedding Layer:** Maps discrete integer event identifiers to dense d -dimensional embedding vectors. Positional encodings are added to preserve sequence order information.
- 2) **Multi-Head Self-Attention:** H parallel attention heads each compute scaled dot-product attention over the full sequence:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

The H head outputs are concatenated and projected, enabling the model to simultaneously attend to events at different relative positions.

- 3) **Layer Normalization and Residual Connections:** Applied after each attention and feed-forward sublayer to stabilize training dynamics.
- 4) **Feed-Forward Sublayer:** A position-wise two-layer MLP with ReLU activation expands and contracts the representation dimensionality.
- 5) **Anomaly Classification Head:** The pooled sequence representation is passed through a sigmoid-activated

output neuron that produces a binary anomaly score $\hat{y} \in [0, 1]$.

C. Domain Adaptation via Gradient Reversal Layer

To address the domain shift problem, a Gradient Reversal Layer (GRL) is inserted between the shared feature extractor F_θ and a domain classification head D_ϕ [4]. The GRL passes its input unchanged during the forward pass, but multiplies the backpropagated gradient by $-\lambda$ during the backward pass:

$$\text{GRL}(\mathbf{x}) = \mathbf{x}, \quad \frac{\partial \mathcal{L}_D}{\partial \mathbf{x}} \longleftarrow -\lambda \cdot \frac{\partial \mathcal{L}_D}{\partial \mathbf{x}}$$

The combined training objective is:

$$\mathcal{L} = \mathcal{L}_{cls} - \lambda \cdot \mathcal{L}_{dom}$$

where $\mathcal{L}_{cls}$ is the binary cross-entropy anomaly classification loss, $\mathcal{L}_{dom}$ is the domain classification cross-entropy loss, and λ is a trade-off hyperparameter that is progressively increased during training. The adversarial domain loss forces F_θ to produce representations that are maximally discriminative for anomaly detection yet minimally informative for domain identification — the definition of domain-invariant features.

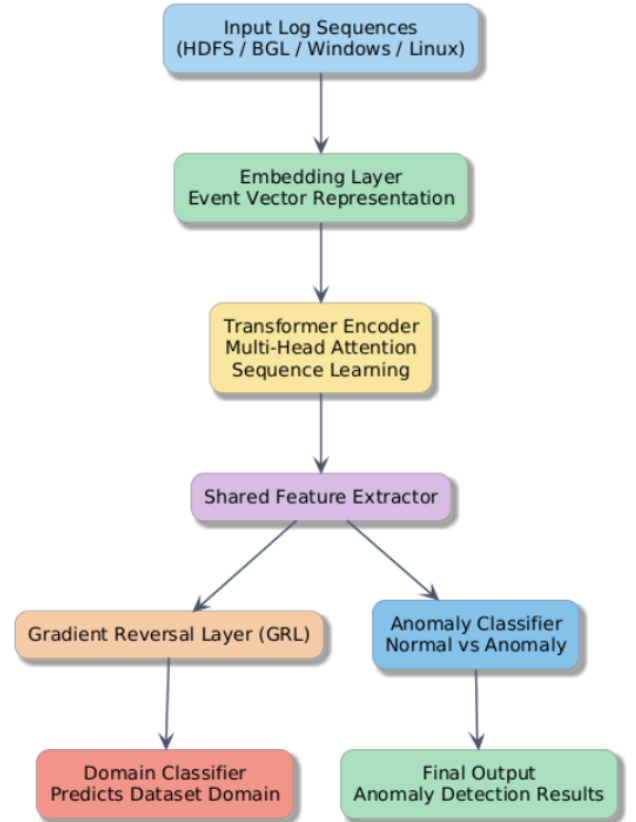


Fig. 4. Domain-Adaptive Transformer architecture integrating a Gradient Reversal Layer (GRL) for cross-system log anomaly detection. The shared encoder produces features consumed by both the anomaly classifier and the domain discriminator

D. Cross-Domain Experimental Configurations

The following transfer learning experiments were conducted:

- **HDFS → BGL:** Source domain HDFS; target domain BGL.
- **HDFS → Windows:** Source domain HDFS; target domain Windows event logs.
- **Multi-domain Training:** Joint training on HDFS, BGL, and Windows with GRL-based domain adaptation, evaluating each domain as the held-out test set.

VII. EXPERIMENTAL RESULTS

A. Linux-Based Model Evaluation

Table II presents the evaluation of the three detection modules on the Linux authentication log dataset.

TABLE II
PERFORMANCE OF LINUX-BASED ANOMALY DETECTION MODELS

Model	Accuracy	Precision	Recall
ML-Only (Frequency)	0.715	0.214	0.068
Rule-Based Only	0.920	1.000	0.668
Hybrid (ML + Rules)	0.876	0.745	0.737

1) *ML-Only Model:* The frequency-based sequence learning model achieved an accuracy of 0.715 but with notably low precision (0.214) and recall (0.068). While the model successfully identified frequently recurring normal sequences, it struggled to correctly classify rare or previously unseen anomalous patterns. The fundamental limitation of frequency-based learning is that it cannot flag sequences that resemble normal behaviour in structure but differ in semantic intent.

2) *Rule-Based Model:* The rule-based module achieved the highest accuracy (0.920) and perfect precision (1.000), confirming that cybersecurity rules produce highly reliable and deterministic alerts with zero false positives when applied to known attack patterns. However, the recall of 0.668 indicates that a significant proportion of anomalous events (those not matching any hardcoded rule) were missed. This confirms the inherent limitation of static signature-based systems when confronted with novel or polymorphic attacks.

3) *Hybrid Model:* The hybrid framework produced the most balanced performance profile: accuracy of 0.876, precision of 0.745, and recall of 0.737. The fusion of ML adaptability and rule-based reliability substantially reduced the false-negative rate compared to the rule-based-only model, while preserving far higher precision than the ML-only model.

B. Transformer-Based Cross-Domain Results

Table III summarizes the F1-scores obtained in cross-domain transfer experiments.

TABLE III
CROSS-DOMAIN TRANSFER EXPERIMENT RESULTS

Model	Setup	F1-Score	Observation
Plain Transformer	HDFS → BGL	0.15	Domain shift failure
Regularized Transformer	HDFS → BGL	0.20	Minor improvement
Domain Adaptive (GRL)	Multi-domain	0.93	Strong generalization
Domain Adaptive (GRL)	HDFS → Windows	0.97	Best result achieved

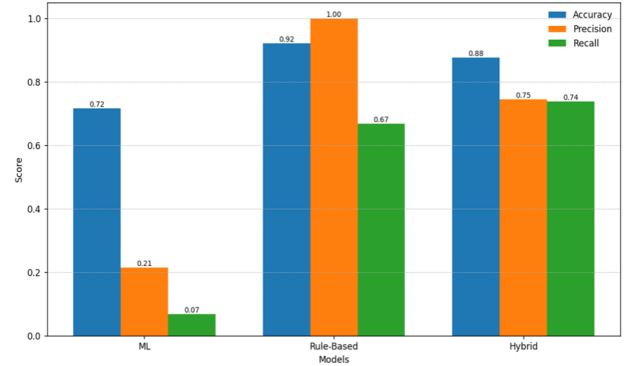


Fig. 5. Comparative performance of ML, rule-based, and hybrid anomaly detection models across accuracy, precision, and recall

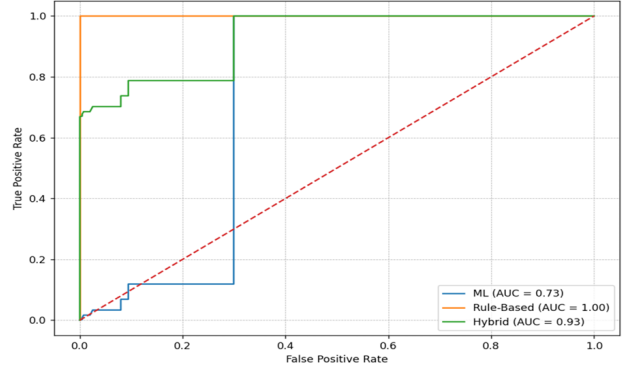


Fig. 6. ROC curve comparison across ML-only, rule-based, and hybrid anomaly detection models

The plain Transformer trained exclusively on HDFS logs achieved an F1-score of only 0.15 when evaluated on BGL logs, confirming the severe impact of domain shift on direct model transfer. Adding weight decay regularization provided only marginal improvement (0.20). In contrast, the GRL-augmented domain-adaptive Transformer achieved F1-scores of 0.93 (multi-domain) and 0.97 (HDFS to Windows), demonstrating that adversarial domain adaptation substantially mitigates the negative transfer problem even across highly heterogeneous log environments.

VIII. DISCUSSION

The experimental results yield several important insights regarding the design of robust log anomaly detection systems.

Limitations of statistical sequence learning. Frequency-based models learn the empirical distribution of event n-grams and flag statistical outliers. While effective for detecting rare or unusual sequences in a closed and stable environment, such models are highly sensitive to class imbalance (normal sequences dominate the training distribution) and cannot generalize to sequences that have never been observed but are semantically anomalous. The very low recall of the ML-only model (0.068) in the Linux evaluation validates this limitation.

Precision-recall trade-off in rule-based systems. The perfect precision (1.000) achieved by the rule-based module

confirms that manually encoded cybersecurity rules are highly reliable for known attack signatures. However, the recall ceiling imposed by the finite signature set is a fundamental constraint. Without continuous signature updates by security analysts, rule-based systems degrade in coverage as threat actors evolve their tactics.

Hybrid fusion for balanced performance. The hybrid framework demonstrates that the complementary strengths of statistical and deterministic detection can be exploited through a simple logical-OR fusion strategy. The recall improvement from 0.668 (rule-based only) to 0.737 (hybrid) confirms that the ML module captures anomalous patterns that rules do not encode, while the precision reduction from 1.000 to 0.745 reflects the noisier nature of frequency-based flagging.

Domain shift as the central challenge in cross-system deployment. The cross-domain experiments provide the most consequential finding of this work: a Transformer model achieving strong performance on HDFS logs collapses to near-random performance ($F1 = 0.15$) when directly applied to BGL logs. This confirms that domain shift is not a marginal concern but a fundamental deployment barrier for log anomaly detection systems in heterogeneous infrastructure.

GRL-based domain adaptation as an effective solution. The GRL mechanism operationalizes a theoretically well-grounded solution by making domain information adversarially uninformative to the shared feature extractor. The resulting features characterize anomalous behaviour in a manner that is agnostic to the source system, enabling effective transfer. The F1-score of 0.97 in the HDFS $\rightarrow$ Windows experiment — across arguably the most dissimilar pair of datasets in this study — strongly validates the effectiveness of this approach.

Importance of unified preprocessing. The unified preprocessing pipeline, particularly the globally fitted LabelEncoder, also played a significant role in enabling cross-domain learning. Without a consistent event vocabulary across domains, the embedding layer would produce incompatible representations for events that are semantically related but lexically different. The normalization of noise tokens (`<NUM>`, `<IP>`) further reduced spurious domain-specific variance in the feature space.

IX. LIMITATIONS

This work has several limitations that should be acknowledged:

- 1) **Heuristic Anomaly Labeling:** For some experimental subsets, anomaly labels were assigned using heuristic rules rather than ground-truth annotations, which may introduce label noise.
- 2) **Dataset Class Imbalance:** All four datasets exhibit significant class imbalance with anomalous sequences forming a small minority. Although standard evaluation metrics were used, imbalance may still influence threshold selection.
- 3) **No Real-Time Deployment:** All experiments were conducted offline on stored log files. Real-time streaming detection introduces additional engineering challenges not addressed here.

- 4) **Frequency-Based ML Scope:** The ML module is limited to frequency-based n-gram modelling. More expressive supervised or self-supervised models were not evaluated in the Linux-specific component.
- 5) **Partial Domain Overlap:** Some HDFS-to-Windows cross-domain experiments may have benefited from partial vocabulary overlap introduced during global label encoding, which could marginally inflate reported transfer performance.

X. FUTURE WORK

Several directions present themselves for future investigation:

- **Real-Time Streaming Detection:** Integration with Apache Kafka or Fluentd for online, low-latency log anomaly detection at production scale.
- **Self-Supervised Pre-Training:** Leveraging contrastive self-supervised objectives (e.g., SimCLR adapted for event sequences) to learn transferable log representations without labelled data.
- **Transformer-LSTM Hybrid:** Combining the long-range dependency modelling of Transformers with the inductive bias of LSTMs for sequential log modelling.
- **SIEM Integration:** Deploying the framework as a plugin within Security Information and Event Management systems such as Splunk or Elastic SIEM.
- **Federated Anomaly Detection:** Exploring privacy-preserving federated learning protocols that enable cross-organization model training without sharing raw log data.
- **Cloud-Native Deployment:** Containerizing the framework using Docker and deploying inference endpoints via Kubernetes for elastic scaling in cloud environments.
- **Industrial-Scale Evaluation:** Evaluating the framework on production-scale log datasets containing billions of events to assess computational scalability and detection latency.

XI. CONCLUSION

This paper presented a unified hybrid and domain-adaptive framework for cross-system log anomaly detection. Beginning with foundational concepts of DeepLog-inspired sequence learning on HDFS logs, the work progressively incorporated rule-based cybersecurity detection, hybrid fusion, Transformer-based contextual modelling, and Gradient Reversal Layer-based domain adaptation to address the increasingly complex challenges of large-scale and heterogeneous log analysis.

The Linux-based practical implementation demonstrated that real-world cybersecurity anomalies — including brute-force attacks, user enumeration, and privilege escalation — can be reliably identified by combining statistical sequence analysis with deterministic rule engines. The hybrid framework achieved the most balanced trade-off between precision and recall among the Linux-based models.

The Transformer-based cross-domain experiments provided a definitive demonstration that domain shift is a critical and practically significant challenge for log anomaly detection

deployments spanning heterogeneous infrastructure. The GRL-augmented domain-adaptive Transformer successfully overcame this challenge, achieving F1-scores of up to 0.97 in the most challenging cross-system transfer experiments — a dramatic improvement over the 0.15 achieved by a plain Transformer without domain adaptation.

The results collectively affirm that robust, scalable, and generalizable log anomaly detection for modern heterogeneous computing environments requires the integration of multiple complementary paradigms: statistical learning for adaptability, rule-based reasoning for reliability, deep learning for contextual understanding, and domain adaptation for cross-system transfer. The proposed framework provides a strong foundation for future research and practical deployment in cybersecurity monitoring and fault-tolerant system management.

REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298.
- [2] W. Meng *et al.*, “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs,” in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 4739–4745.
- [3] H. Guo *et al.*, “LogBERT: Log Anomaly Detection via BERT,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021.
- [4] Y. Ganin *et al.*, “Domain-Adversarial Training of Neural Networks,” *J. Mach. Learn. Res.*, vol. 17, no. 1, pp. 2096–2030, 2016.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [6] J. Zhu *et al.*, “Loghub: A Large Collection of System Log Datasets for AI-Driven Log Analytics,” *arXiv preprint arXiv:2008.06448*, 2023.
- [7] A. Oliner and J. Stearley, “What Supercomputers Say: A Study of Five System Logs,” in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2007, pp. 575–584.
- [8] M. Abadi *et al.*, “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” *arXiv preprint arXiv:1603.04467*, 2016.
- [9] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguist. (NAACL-HLT)*, 2019, pp. 4171–4186.